Experiment No.5

Code :

```
from collections import deque
GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)
def get_neighbors(state):
    neighbors = []
    i = state.index(0)          # position of blank
    row, col = i // 3, i % 3
    moves = [(-1,0), (1,0), (0,-1), (0,1)]  # up, down, left, right
    for dr, dc in moves:
        r, c = row + dr, col + dc
        if 0 <= r < 3 and 0 <= c < 3:
            j = r * 3 + c
            new_state = list(state)
            new_state[i], new_state[j] = new_state[j], new_state[i]
            neighbors.append(tuple(new_state))
    return neighbors


def bfs(start):
    visited = {start}
    queue = deque([(start, [])])
    while queue:
        state, path = queue.popleft()
        if state == GOAL:
            return path + [state]
        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [state]))
    return None
```

```
def print_state(state):
    for i in range(0, 9, 3):
        print(state[i:i+3])
    print()
if __name__ == "__main__":
    start = (5, 4, 1, 8, 6, 2, 7, 3)   # 0 = blank tile
    solution = bfs(start)
    if solution:
        print("Solved in", len(solution) - 1, "moves\n")
        for step, state in enumerate(solution):
            print("Step", step)
            print_state(state)
    else:
        print("No solution found")
```

Output:

```
Solved in 2 moves
Step 0
(1, 2, 3)
(4, 5, 6)
(0, 7, 8)

Step 1
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)
```

Step 2

(1, 2, 3)

(4, 5, 6)

(7, 8, 0)

Experiment No.6

Code:

```
CAP_A = 9      # capacity of jug A
CAP_B = 8      # capacity of jug B
GOAL = 7       # amount we want in either jug

def get_neighbors(state):
    a, b = state
    neighbors = []

    neighbors.append((CAP_A, b))            # fill A
    neighbors.append((a, CAP_B))            # fill B
    neighbors.append((0, b))                # empty A
    neighbors.append((a, 0))                # empty B

    pour = min(a, CAP_B - b)                # pour A -> B
    neighbors.append((a - pour, b + pour))

    pour = min(b, CAP_A - a)                # pour B -> A
    neighbors.append((a + pour, b - pour))

    return neighbors

def dfs(start):
    visited = set()
    stack = [(start, [start])]

    while stack:
```

```
        state, path = stack.pop()
        if state[0] == GOAL or state[1] == GOAL:
            return path

        if state in visited:
            continue
        visited.add(state)
        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                stack.append((neighbor, path + [neighbor]))

    return None

if __name__ == "__main__":
    start = (0, 0)   # both jugs empty
    solution = dfs(start)

    if solution:
        print("Solution found:\n")
        for step, state in enumerate(solution):
            print("Step", step, "-> Jug A:", state[0], ", Jug B:", state[1])
    else:
        print("No solution found")
```

Output:

Solution found:

Step 0 -> Jug A: 0 , Jug B: 0

Step 1 -> Jug A: 0 , Jug B: 8

Step 2 -> Jug A: 8 , Jug B: 0

Step 3 -> Jug A: 8 , Jug B: 8

Step 4 -> Jug A: 9 , Jug B: 7

Experiment No.7

Code :

```
import re
rules = [
    (r'hi|hello|hey', ['Hello! How can I help you today?', 'Hi there!']),
    (r'my name is (.*)', ['Nice to meet you, {0}!']),
    (r'how are you', ["I'm doing well, thank you! How about you?"]),
    (r'what is your name', ['I am a simple rule based chatbot.']),
    (r'(.*) help (.*)', ['Sure, tell me more about your problem.']),
    (r'bye|quit|exit', ['Goodbye! Have a nice day.']),
]

def get_response(user_input):
    user_input = user_input.lower().strip()
    for pattern, responses in rules:
        match = re.search(pattern, user_input)
        if match:
            response = responses[0]
            if match.groups():
                response = response.format(*match.groups())
            return response
    return "I'm sorry, I did not understand that. Can you rephrase?"

if __name__ == "__main__":
    print("Chatbot: Hello! Type 'bye' to exit.")
    while True:
        user_input = input("You: ")
        reply = get_response(user_input)
        print("Chatbot:", reply)
```

```
    if re.search(r'bye|quit|exit', user_input.lower()):
        break
```

Output:

Chatbot: Hello! Type 'bye' to exit.

You: hii

Chatbot: Hello! How can I help you today?

You: what is your name

Chatbot: I am a simple rule based chatbot.

You: test

Chatbot: I'm sorry, I did not understand that. Can you rephrase?

You: bye

Chatbot: Goodbye! Have a nice day.

Experiment No.8

Code:

```
import math


# Initialize the board
board = [" " for _ in range(9)]
def print_board():
  print()
  print(f" {board[0]} | {board[1]} | {board[2]} ")
  print("-----------")
  print(f" {board[3]} | {board[4]} | {board[5]} ")
  print("-----------")
  print(f" {board[6]} | {board[7]} | {board[8]} ")
  print()


def check_winner(b, player):
  # Winning combinations: rows, columns, diagonals
  win_conds = [
     (0, 1, 2),
     (3, 4, 5),
     (6, 7, 8),  # rows
     (0, 3, 6),
     (1, 4, 7),
     (2, 5, 8),  # columns
     (0, 4, 8),
     (2, 4, 6),  # diagonals
  ]
  for cond in win_conds:
    if b[cond[0]] == b[cond[1]] == b[cond[2]] == player:
```

```
      return True
  return False

def is_board_full(b):
  return " " not in b
def get_available_moves(b):
  return [i for i, spot in enumerate(b) if spot == " "]

# Minimax Algorithm Implementation
def minimax(b, depth, is_maximizing):
  # Base cases: check for terminal states
  if check_winner(b, "O"):  # AI wins
    return 10 - depth
  if check_winner(b, "X"):  # Human wins
    return depth - 10
  if is_board_full(b):  # Tie
    return 0

  if is_maximizing:
    max_eval = -math.inf
    for move in get_available_moves(b):
      b[move] = "O"
      eval = minimax(b, depth + 1, False)
      b[move] = " "
      max_eval = max(max_eval, eval)
    return max_eval
  else:
    min_eval = math.inf
    for move in get_available_moves(b):
```

```
      b[move] = "X"
      eval = minimax(b, depth + 1, True)
      b[move] = " "
      min_eval = min(min_eval, eval)
    return min_eval

# Find the best move for the AI
def ai_move():
  best_val = -math.inf
  best_move = None
  for move in get_available_moves(board):
    board[move] = "O"
    move_val = minimax(board, 0, False)
    board[move] = " "
    if move_val > best_val:
      best_val = move_val
      best_move = move
  return best_move

# Main Game Loop
def play_game():
  print("Welcome to Tic-Tac-Toe! You are 'X' and the AI is 'O'.")
  print("Board positions are numbered from 0 to 8 like this:")
  print(" 0 | 1 | 2 ")
  print("-----------")
  print(" 3 | 4 | 5 ")
  print("-----------")
  print(" 6 | 7 | 8 \n")
```

```
while True:
  print_board()
  # Human Turn ('X')
  try:
    move = int(input("Enter your move (0-8): "))
    if board[move] != " ":
      print("Spot is already taken! Choose another.")
      continue
  except (ValueError, IndexError):
    print("Invalid input. Please enter a number between 0 and 8.")
    continue

  board[move] = "X"

  if check_winner(board, "X"):
    print_board()
    print("Congratulations! You beat the unbeatable AI (This shouldn't happen!).")
    break
  if is_board_full(board):
    print_board()
    print("It's a tie!")
    break
  # AI Turn ('O')
  print("AI is thinking...")
  ai_choice = ai_move()
  board[ai_choice] = "O"
  if check_winner(board, "O"):
    print_board()
    print("AI wins! Better luck next time.")
```

```
      break
    if is_board_full(board):
      print_board()
      print("It's a tie!")
      break

if __name__ == "__main__":
  play_game()
```

Output:

Welcome to Tic-Tac-Toe! You are 'X' and the AI is 'O'.

Board positions are numbered from 0 to 8 like this:

```
 0 | 1 | 2
-----------
 3 | 4 | 5
-----------
 6 | 7 | 8
```

```
   |   |
-----------
   |   |
-----------
   |   |
```

Enter your move (0-8): 2

AI is thinking...

```
   |   | X
-----------
```

| O |

| |

Enter your move (0-8): 4

Spot is already taken! Choose another.

| | X

| O |

| |

Enter your move (0-8): 5

AI is thinking...

| | X

| O | X

| | O

Enter your move (0-8): 0

AI is thinking...

X | O | X

| O | X

| | O

Enter your move (0-8): 7

AI is thinking...

X | O | X

O | O | X

  | X | O

Enter your move (0-8): 6

X | O | X

O | O | X

X | X | O

It's a tie!